Task 2

The efficient algorithms are the core for solving complex problems. Two important ideas that helps in explaining challenges are the concept of P vs NP and heuristic methods. This idea is used in algorithm design, cryptography and artificial intelligence. In this section we will cover: P vs NP explanation, Brute force approach and Heuristic approach

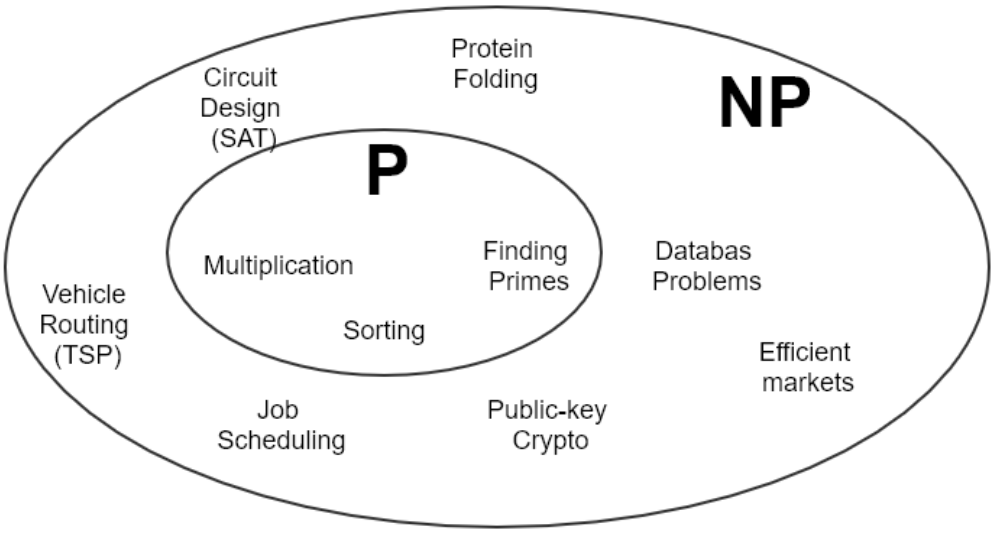

Figure 25: P and NP problem (Src: ruyasoft)

P problems

P problems refer to the problems that can be solve in the polynomial amount of time. Deterministic Turning Machine takes polynomial time to solve this type of problem. For example, everyday problems like:

- All the basic mathematical operations like addition, subtraction, division, etc
- Sorting name in alphabetical order
- Searching for data
- Finding the shortest path

NP (non-deterministic polynomial) problems

NP problems are that problem which cannot be solved in polynomial time but can be verified in polynomial time polynomial. For example, a Sudoku game

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figure 26: Sudoku

Solving sudoku takes lot of time. However, if solved sudoku is given, it would be much less complex to check if there are any repetition in the rows, columns and matrices. This task is a simple which would have a polynomial run-time.

Seating scenario:

Why checking a seating plan is easy?

Assume that the teacher is already given a complete seating arrangement of all students. To verify this arrangement teacher only need to verify student who are sitting next to each other. For each pair of students, teacher checks weather they are friends or not and ensures that friends are not sitting side by side. In addition, teacher also checks weather pair of students are from the same city. The checking time for comparison is limited and requires examining only neighbouring seats rather than the entire arrangements. Therefore, verifying a given siting plan can be done efficiently in short amount of time. It is computationally simple.

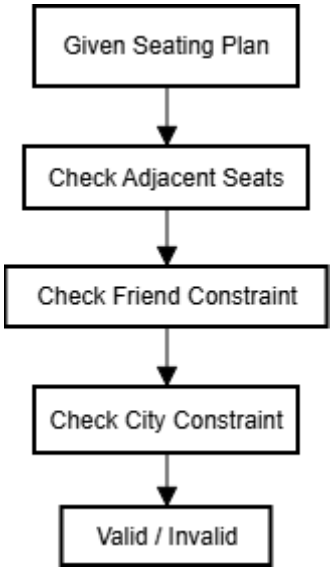

Figure 27: Verifying Seating Plan

Why finding a seating plan is difficult?

In the beginning, there is no valid sitting arrangements. The teacher must determine correct order from scratch. To find the suitable arrangement, teacher must consider all possible seating permutations of students as the possible solution. The number of permutations increases factorially as the number of students grows. This causes the problem size to increase very rapidly. For example,

If we want to arrange seats for 5 students, we can arrange them in 120 different ways. Similarly, if we want to arrange 10 students, we can arrange then in 3628800 different ways. This can be calculated as n! where n is the number of people.

Therefore, finding a valid seat arrangement is more computationally challenging than checking the seating plan.

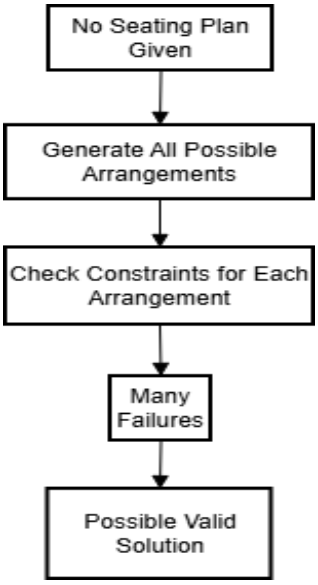

Figure 27: Finding seat plan

Seating arrangement is P or NP problem

Seating arrangement problem is closer to NP problem rather than p because while seating arrangement verification is quick by checking adjacent student but finding an arrangement without prior information is difficult. The problem becomes much harder because of the rules which must be followed at the same time. There is no valid algorithm that can guarantee finding valid seating arrangement efficiently for given problem. Therefore, the problem demonstrates the key characteristics of NP problem, where solution verification is simple but finding solution is computationally complex.

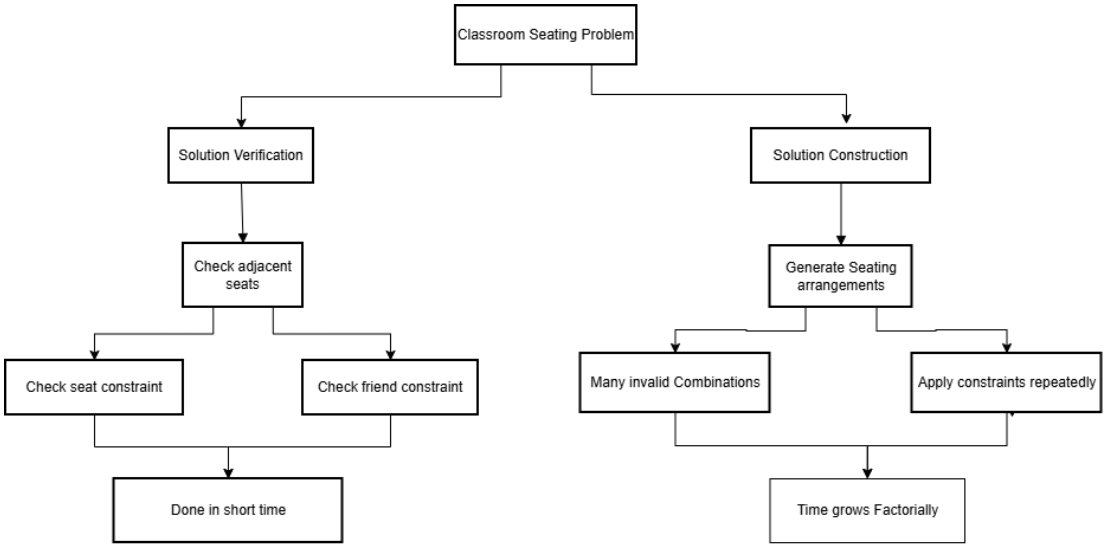

Figure 28: Classroom Seating problem

Figure 28 illustrates why the classroom seating arrangement problem is closer to NP. Solution verification requires only requires only local checks. Whereas constructing a valid seat arrangement requires exploring many possible combinations with multiple constraints.

Brute force Approach

Brute force algorithm solves the problem by trying all potential solutions and checking each one until a valid solution is found. It is a simplest method, but this approach is inefficient. It provides a guaranteed solution to the problem, but it requires lot of time as the size of the problem increases.

For example, in list searching, it checks all elements and find out if searched word is present or not. Below is the simple python brute force approach in name list.

```
Task 2 > Brute FOrce.py > ...
# simple brute force approach

Name = ["Ram", "Shyam", "Hari", "Sita", "Gita", "Laxman", "Rawan", "Nobita", "Sunio", "Gian", "Sizuka",
        "degisugi", "Doreamon", "Doreme", "Sourav", "Puyush", "Sahil", "Avantiva"]

Name_to_search = "Doreamon"

for i in Name:
    if i == Name_to_search:
        print(f"{i} found in list")
    else:
        continue

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS        powershell

PS D:\Course work\Foundation-Work> & C:\Users\Nitro\AppData\Local\Programs\Python\Python313\python.exe "d:/Course work/Found
Work/Task 2/Brute FOrce.py"
Doreamon found in list
PS D:\Course work\Foundation-Work>
```

Figure 29: Brute force approach in list

```
Task 2 > bruteforce_in_findingword.py > ...
# little complex brute force

wrod_to_find = "ram"

alphabet = "abcdefgnijklmnopqrstuvwxyt"

for i in alphabet:
    for j in alphabet:
        for k in alphabet:
            combined = i+j+k
            if combined == wrod_to_find:
                print(f"found word {combined}")
                break

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS        powershell

PS D:\Course work\Foundation-Work> & C:\Users\Nitro\AppData\Local\Programs\Python\Python313\python.exe "d:/Course work/Foundation-
Work/Task 2/bruteforce_in_findingword.py"
found word ram
PS D:\Course work\Foundation-Work>
```

Figure 30: Brute Force approach in finding word

In figure 30, we can see that, to find word ram we need three loop that means 26*26*26 which is equal to 17576 words are created to find the word ram. Similarly, if the data is long like 10 characters, then there will be 141167095653376 words to check. Therefore, this is impractical in real-life where time and resource is limited.

Brute force method for Classroom seating arrangement problem.

We already checked that brute force is effective for small. So, in seating arrangement when the number of students is small then brute force approach is effective because the total number of seating arrangement remains limited and computationally manageable. Complexity that arises during arrangement of huge number of students are:

1. Factorial Growth

For example, if there are only 5 students then possible number of seating arrangement is 5! = 120. This can be quickly evaluate using computer program. However, as the number of student

increases, possible arrangements also increase. The total number of combinations multiplies rapidly.

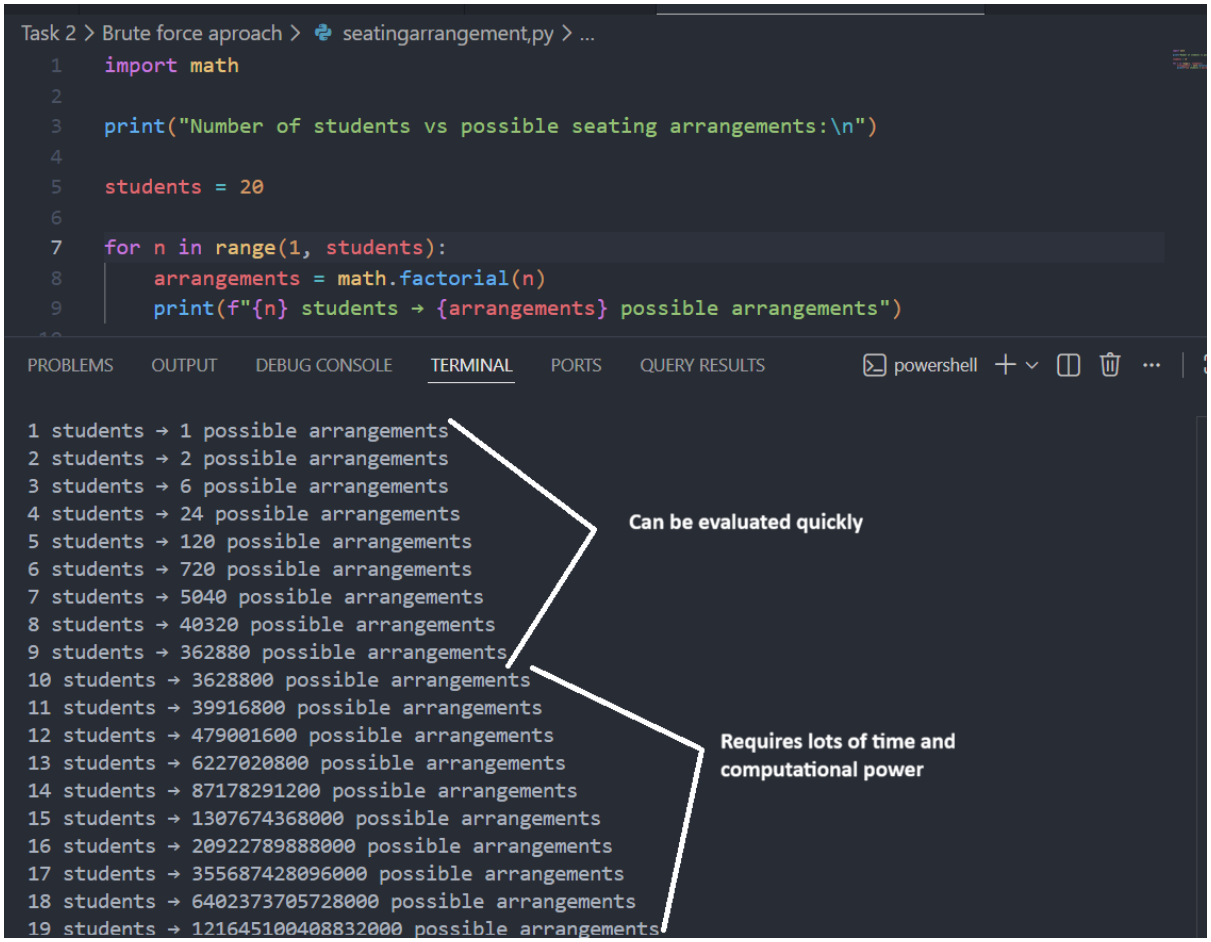

Figure 31: Number of students vs possible seating arrangement

From the figure 31 we can see that as the student increases number of possible arrangements grows factorially. That means each additional student multiplies the total number of combinations.

2. Time complexity

```
Task 2 > Brute force aproach > time complexity.py > ...
import math

check_time = 0.000001  # assuming 1 microsecond per arrangement

print("Estimated time to check all arrangements:\n")

last_student = 20
for n in range(5, last_student):
    total_time = math.factorial(n) * check_time
    print(f"{n} students → {total_time:.2f} seconds")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS        powershell

PS D:\Course work\Foundation-Work> & C:\Users\Nitro\AppData\Local\Programs\Python\Python313\python.exe "d:/Course work/Foundation-Wo
sk 2/Brute force aproach/time complexity.py"
Estimated time to check all arrangements:

5 students → 0.00 seconds
6 students → 0.00 seconds
7 students → 0.01 seconds
8 students → 0.04 seconds
9 students → 0.36 seconds
10 students → 3.63 seconds
11 students → 39.92 seconds
12 students → 479.00 seconds
13 students → 6227.02 seconds
14 students → 87178.29 seconds
15 students → 1307674.37 seconds
16 students → 20922789.89 seconds
17 students → 355687428.10 seconds
18 students → 6402373705.73 seconds
19 students → 121645100408.83 seconds
PS D:\Course work\Foundation-Work>
```

Figure 32: Time for arrangement

From figure 32, we can see that it is suitable to apply brute force approach for up to 9 students it takes less than 1 sec. But when number of students grows from that point time increases rapidly. When there will be 15 students it takes 15 days and when there will be 19 students it takes 1407929.4 days. It shows the complexity arises in time. Therefore, it is impracticable for solutions.

Therefore, due to factorial growth in arrangement and time growth, it is difficult for solving this problem. Even modern computer with high processing power struggle to evaluate all possible arrangements within a time. This is a main reason why the seating arrangement problem shows traits of NP problems. These problems are easy to verify but hard to find the solution.

Heuristics approach.

Heuristics methods are the strategies that aim to find reasonably good solutions efficiently. They are commonly used for computationally difficult problem like NP-type problem. Heuristics apply logical rules or informed guesses to guide decision-making. Heuristics Methods can be used for following reasons.

1. Reduce Complexity

Heuristics methods can be used to reduce the complexity of finding a valid seating arrangement while satisfying the constraints. A good way to start is by seating students who have lots of friends first. These students have more rules to follow about who they can sit next to. Putting them in their seats early helps avoid conflict. Another step is to separate the student from the

same city early in the starting process. It also makes it easier to place everyone else without having to go back and change things.

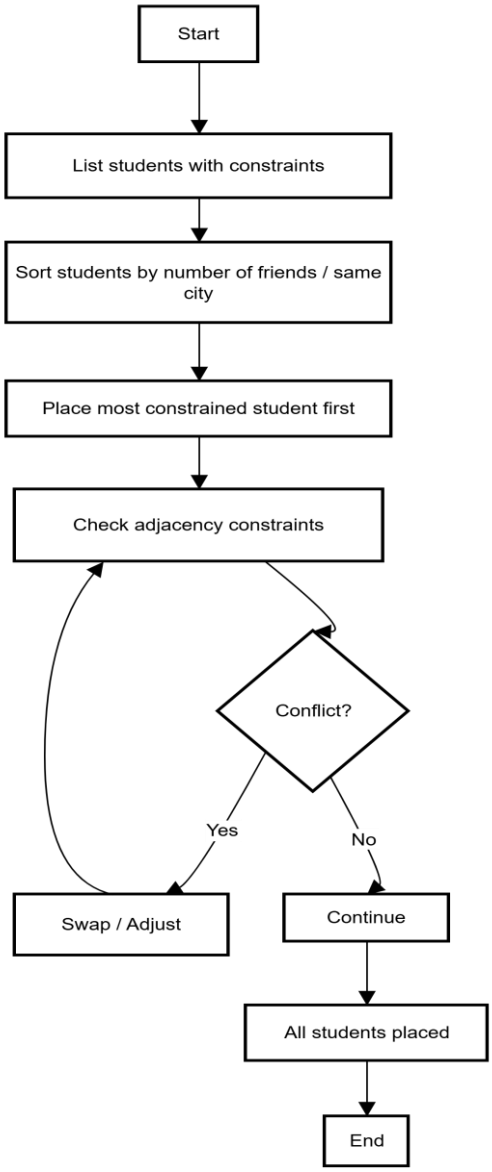

Figure 33: Heuristic Seating Workflow Diagram

2. Save Time

Heuristic methods save time mainly by narrowing down the options to consider. Another benefit of using heuristics is catching problems early. For example, seating students with the most restrictions first or keeping students from the same city apart helps spot issues right away.

Compared to brute force methods that check every option no matter what, heuristics cut down the number of possibilities a lot. This makes finding a solution much faster, especially with more students involved.

Solving Classroom arrangement problem.

Let's assume that there are 16 students. We found out that from brute force there will be 2092278988800 possible arrangements which requires 20922789.89 second which is equal to 242 days to evaluate so it is not efficient. So, from Heuristic methods this problem can be solved.

```python
students = ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P"]

# Friendship
friends = {
    "A": ["B", "C", "D"],
    "B": ["A", "E"],
    "C": ["A", "F"],
    "D": ["A", "G"],
    "E": ["B", "H", "I"],
    "F": ["C", "J"],
    "G": ["D", "K"],
    "H": ["E"],
    "I": ["E", "L"],
    "J": ["F", "M"],
    "K": ["G", "N"],
    "L": ["I"],
    "M": ["J"],
    "N": ["K"],
    "O": ["P"],
    "P": ["O"]
}


city = {
    "A": "Kathmandu", "B": "Pokhara",   "C": "Kathmandu", "D": "Kathmandu",
    "E": "Pokhara",   "F": "Biratnagar", "G": "Lalitpur",  "H": "Pokhara",
    "I": "Chitwan",   "J": "Biratnagar", "K": "Lalitpur",  "L": "Chitwan",
    "M": "Biratnagar","N": "Butwal",     "O": "Dhangadhi","P": "Dhangadhi"
}
```

Figure 34: Student information

Using Heuristic method to find the sit arrangement of this student

```
PS D:\Course work\Foundation-Work> & C:\Users\Nitro\AppData\Local\Programs\Python\Python313\python.exe "d:/Course work/Foundation-Work/Ta
sk 2/heuristic aproach/heuristic.py"
Order of placement (most constrained first):
 → E A F J K G C D I B N L M H O P

✓ Placed  E  →  E
✓ Placed  A  →  A → E
✓ Placed  F  →  A → F → E
✓ Placed  J  →  J → A → F → E
✓ Placed  K  →  J → A → K → F → E
✓ Placed  G  →  J → G → A → K → F → E
✓ Placed  C  →  J → C → G → A → K → F → E
✓ Placed  D  →  J → C → G → A → K → D → F → E
✓ Placed  I  →  J → C → G → A → I → K → D → F → E
✓ Placed  B  →  J → C → G → A → I → B → K → D → F → E
✓ Placed  N  →  J → C → G → A → I → N → B → K → D → F → E
✓ Placed  L  →  J → C → G → A → I → N → L → B → K → D → F → E
✓ Placed  M  →  J → C → G → A → I → N → M → L → B → K → D → F → E
✓ Placed  H  →  J → C → G → A → I → N → H → M → L → B → K → D → F → E
✓ Placed  O  →  J → C → G → A → I → N → H → O → M → L → B → K → D → F → E
✓ Placed  P  →  J → C → G → A → I → N → P → H → O → M → L → B → K → D → F → E

═══════════════════════════════════════════════════════════
SUCCESS! Final valid seating arrangement found:
 → J → C → G → A → I → N → P → H → O → M → L → B → K → D → F → E
PS D:\Course work\Foundation-Work>
```

Figure 35: Seat arrangement using heuristic method

From the figure 35 we can see that using heuristic method, it is very easy to find the seat arrangement. This approach saves time and complexity. Such approaches reflect real-world computing practices.

Despite its efficiency, it does not guarantee a perfect solution. The effectiveness of a heuristic methods depends on early decisions. If initial strategy is poorly chosen, then it will affect result.

Conclusion of Classroom Seating Arrangement Problem

We have covered Brute force method and Heuristics method to find the solution. After analysing both methods, we can conclude that for arrangement problem Heuristics method is best for the solution as it solves time complexity and provide a required solution. Brute force provides a perfect solution though it is in efficient for larger problems.